Catalytic branching programs trade polynomial register space for exponential-time grouping costs

James Cook, Ian Mertz

doi:10.4230/LIPIcs.CCC.2022.8; CC BY 4.0

Editorial note. Complete official published author models, formulas, subroutines and proofs transcribed; formal mainTheorems11/15 grouped into one tradeoff record. Original source algorithm references and expressions retained without mathematical corrections. Long register instructions only split across display lines for page width. Frozen dependencies of earlierCC040 unchanged. All production and validation run only onYupeng viaSSH.

Allowed standard prerequisites (editorial).

- Multilinear Boolean polynomial interpolation and cyclic Gray codes. Source-local interpolation proof fully retained; original Gray-code enumeration is the standard cited ingredient9. All register transparency and branching-program conversion arguments included.

2 Preliminaries

We introduce two space-bounded models of computation. Our first model is a variation of branching programs, which are the standard syntactic (i.e. non-uniform) notion of space-bounded computation (see [7]).

Definition5 (m-catalytic branching program [8]). Let $n \in \mathbb{N}$ and let $f : \{0,1\}^n \to \{0,1\}$ be an arbitrary function. An m-catalytic branching program is a directed acyclic graph G with the following properties:

- There are m source nodes and $2m$ sink nodes.
- Every non-sink node is labeled with an input variable x_i for $i \in [n]$, and has two outgoing edges labeled0 and1.
- For every source node v there is one sink node labeled with $(v, 0)$ and one with $(v, 1)$.

We say that G computes f if for every $x \in \{0,1\}^n$ and source node v, the path defined by starting at v and following the edges labeled by the value of the x_i labeling each node ends at the sink labeled by $(v, f(x))$. The size of G is

the number of non-sink nodes[1] in G. We also consider m-catalytic branching programs with standard restrictions:

- layered branching programs: for some $\ell \in \mathbb{N}$, there exists a function $\sigma : G \to [\ell+1]$ such that for all $u \in G$, the outgoing edges of u go to nodes v, w such that $\sigma(v) = \sigma(w) = \sigma(u) + 1$; we call the set of nodes $\{v \in \sigma^{-1}(j)\}$ the jth layer. Furthermore for each $j \in [\ell]$ there exists an input variable x_{j_i} which is the variable labeling every $v \in \sigma^{-1}(j)$.[2] The width of G is $\max_{j\in[\ell]} |\sigma^{-1}(j)|$ and the length of G is ℓ. Note that the size of G is at most the product of the length and width of G.

- read-k branching programs: for any start node v and any input x, each variable x_i is read at most k times during the computation starting at v. We note that for layered branching programs, this corresponds to each variable x_j labeling at most k layers.

- permutation branching programs: consider a layered branching program of width $2m$ with $2m$ source nodes $S = \{s_1 \dots s_{2m}\}$ and $2m$ sink nodes $T = \{t_1 \dots t_{2m}\}$, and let $P(x) : [2m] \to [2m]$ be the function such that $P(x)(i) = i'$ where the computation path starting from s_i on input x goes to $t_{i'}$. Then there exists a permutation $\sigma^* : [2m] \to [2m]$ such that $P(x)$ is the identity function when $f(x) = 0$ and $P(x) = \sigma^*$ when $f(x) = 1$. This is not an m-catalytic branching program in the strict sense, and we will address this model more precisely in Section4.

Our second model comes from a line of work starting with [3], more recently fleshed out in [4] and used in many follow-up works on catalytic computation [5,6].

Definition6 (Transparent register program). Let R be a ring and $f \in \{0,1\}^n \to \{0,1\}$ a function. A register program P is defined by a set of registers $\mathcal{S} = \{R_1 \dots R_s, R_{\text{out}}\}$, each storing a value in R, plus an ordered list of t instructions where for every $j \in [t]$ the jth instruction is $R \leftarrow R + p_j(x_i, \mathcal{S} \setminus \{R\})$ for some $i \in [n]$, $R \in \mathcal{S}$, and polynomial $p_j \in R[x, y_0 \dots y_s]$. We say that P computes f if for every $x \in \{0,1\}^n$, after initializing $R = 0$ for all registers R and then executing all instructions in order, the value stored in R_{out} is $f(x)$. We say that P transparently computes f if instead of initializing all R to0, each R begins in an arbitrary initial state τ_i, and at the end of the program we have $R_{\text{out}} = \tau^{\text{out}} + f(x)$ and $R_i = \tau_i$ for all i. The size of P is the number of registers $s + 1$ and the time of P is the number of instructions t. We use register programs as an abstraction for understanding m-catalytic branching programs, which will be the model our main results will refer to. The two models are connected through the following observation (see Figure2 of the original article for an example of our construction).

[1]This is somewhat non-standard, but when talking about layered branching programs this simplifies things by defining the length as the number of times we read variables, which will in turn be connected to the number of instructions in our register program model. This choice does not affect the asymptotics of any results.

[2]By construction layer $\ell + 1$ will contain exactly the sink nodes of G. See the previous footnote for an explanation of this somewhat non-standard convention.

Observation7. Let $f_n : \{0,1\}^n \to \{0,1\}$ be a family of functions and let P_n be a family of register programs over a family of rings R_n with size $s(n)+1$ and time $t(n)$ each transparently computing f_n. Then f_n can be computed by a family of m-catalytic branching programs of width $|R_n|m$ and length $t(n)$, where $m = |R_n|^{s(n)}$.

Proof. We will define a branching program with width $|R_n|^{s(n)+1}$ and length $t(n)$. Each layer will represent a stage in the register program and each node in a layer will represent a setting to the registers at that time. Since each register program step only requires us to read one input variable, at layer k we query the variable associated with step k in the register program and create edges from each node in layer k to the nodes at layer $k+1$ representing the state of our memory after the step has completed. We label each input node as τ for some distinct initial configuration $\tau = (\tau_1 \dots \tau_{s(n)})$ to all registers except R_{out}, and we treat R_{out} as being initialized to0. Then by the fact that P transparently computes f, starting at node τ we are guaranteed to reach a node $(\tau, f(x))$. Since in each layer we have a node for every setting of the $s(n)+1$ registers, the width of our branching program is $|R_n|^{s(n)+1}$, and since each non-output layer corresponds to a unique instruction from our register program, the length of our branching program is $t(n)$. □

Note8. In our computation we often include instructions of the form $R \leftarrow R + p_j(R/\{R\})$, i.e. instructions that do not require reading a variable x_i. By observing the above proof, it is clear that such instructions do not add any length to the branching program, as they can be computed at the same time as an adjacent instruction.

Theorem11. For any $k \in \mathbb{N}$ and any function $f : \{0,1\}^n \to \{0,1\}$ there is an m-catalytic branching program with length $2^k \cdot 2\lceil n/k \rceil$ and width $2m$ that computes f, where $m = 2^{n+k\cdot 2^{\lceil n/k \rceil}}$.

Theorem15. For any function $f : \{0,1\}^n \to \{0,1\}$ which is a degree-d polynomial and any $k \in \mathbb{N}$, there is an m-catalytic branching program with length $2^k \cdot 2n$ and width $2m$ that computes f, where $m = 2^{n+k\cdot\binom{n}{\leq\lceil d/k \rceil}}$.

3 Saving Space

In these sections, our register programs will all operate over the field of two elements: $R_n = \mathbb{F}_2$ for all n.

3.1 Basic Algorithm

In this section, we will prove:

Theorem9. For any function $f : \{0,1\}^n \to \{0,1\}$ there is an m-catalytic branching program with length $4n$ and width $2m$ that computes f, where $m = 2^{n+2^n}$. This is proved by Algorithm1 below. This nearly reproduces Potechin's Theorem3.1[11], but with a worse value $m = 2^{n+2^n}$ instead of 2^{2^n-1}.

Nonetheless, we will find it a useful starting point for our algorithm that trades space for time in Section3.2.

Proof. We will design a program that uses $n+2^n$ work registers plus one output register R_{out}, which is sufficient by Observation7. First, we have n registers $R_1^{\text{in}}, \ldots, R_n^{\text{in}}$, corresponding to the n input bits. This correspondence is given by the following subroutine:

ToggleInput.

1: **procedure** ToggleInput

2: **for** $i = 1, \ldots, n$ **do**

3: $R_i^{\text{in}} \leftarrow R_i^{\text{in}} + x_i$

4: **end for**

5: **end procedure**

After ToggleInput runs, the registers have values $R_i^{\text{in}} = \tau_i^{\text{in}} + x_i$, where τ_i^{in} stands for the initial value of R_i^{in}. If we run it a second time, the registers are restored to their original values: $R_i^{\text{in}} = \tau_i^{\text{in}}$. Since we query all n variables once, ToggleInput requires time n to run once. Before defining our other 2^n registers, we introduce an algebraic view of f, which will be our main focus. We can view f as a multilinear polynomial $p_f \in \mathbb{F}_2[x_1, \ldots, x_n]$ using basic interpolation:

Lemma10. For any function $f : \{0,1\}^n \to \{0,1\}$ there is a multilinear polynomial $p_f \in \mathbb{F}_2[x_1, \ldots, x_n]$ such that $p_f(\vec{x}) = f(\vec{x})$ for all $\vec{x} \in \{0,1\}^n$.

Proof. For any $\vec{y} \in \{0,1\}^n$, we can algebraically define the indicator function $[\vec{x} = \vec{y}]$ as $\prod_{i=1}^n (x_i + y_i + 1) \in \mathbb{F}_2[x_1, \ldots, x_n]$. Then we can set

$$p_f = \sum_{\vec{y} \in \{0,1\}^n : f(\vec{y}) = 1} [\vec{x} = \vec{y}].$$

□

Now define $y_i = \tau_i^{\text{in}} + x_i$; in other words, y_i is the value of R_i^{in} after running ToggleInput. Define $q_f(y_1, \ldots, y_n, \tau_1^{\text{in}}, \ldots, \tau_n^{\text{in}}) = p_f(y_1 - \tau_1^{\text{in}}, \ldots, y_n - \tau_n^{\text{in}})$ to be the result of rewriting p_f using the y_i and τ_i variables.[3] For $S, S' \subseteq [n]$, let $c_{S,S'}$ be the coefficient of $(\prod_{i \in S} \tau_i^{\text{in}}) \cdot (\prod_{i \in S'} y_i)$ in q_f, so that

$$q_f(\vec{\tau}^{\text{in}}, \vec{y}) = \sum_{S, S' \subseteq [n]} c_{S,S'} \left(\prod_{i \in S} \tau_i^{\text{in}} \right) \left(\prod_{i \in S'} y_i \right).$$

Note that S and S' are disjoint whenever $c_{S,S'} \neq 0$, because p_f is multilinear. We now introduce our other registers: we have 2^n registers R_S indexed by subsets $S \subseteq [n]$. Our next subroutine prepares us to use these registers to compute q_f:

[3]For example, if f is the OR function $f(x_1, x_2) = x_1 \vee x_2$, we have $p_f(x_1, x_2) = x_1 + x_2 + x_1 x_2$ and $q_f(\tau_1^{\text{in}}, \tau_2^{\text{in}}, y_1, y_2) = y_1 + \tau_1^{\text{in}} + y_2 + \tau_2^{\text{in}} + y_1 y_2 + y_1 \tau_2^{\text{in}} + \tau_1^{\text{in}} y_2 + \tau_1^{\text{in}} \tau_2^{\text{in}}$, and both are equal to $f(x_1, x_2)$ so long as $\vec{y}$ have the correct values.

ToggleMonomials.

1: **procedure** ToggleMonomials

2: ToggleInput

3: **for** $S' \subseteq [n]$ **do**

4: $R_{S'} \leftarrow R_{S'} + \prod_{i \in S'} R_i^{\text{in}}$

5: **end for**

6: ToggleInput

7: **end procedure**

After ToggleMonomials runs, we have $R_S = \tau_S + \prod_{i \in S} y_i$ for each $S \subseteq [n]$, where τ_S stands for the register's initial value. The R^{in} registers have their initial values $R_i^{\text{in}} = \tau_i^{\text{in}}$. We run ToggleInput twice and have 2^n additional instructions, but since the additional instructions do not query any x variables they can be computed in the last x query of ToggleInput, for a total runtime of $2n$. Our final algorithm for computing f is:

Algorithm1: Transparently compute f in time $4n$ with $n + 2^n + 1$ registers.

1: ToggleMonomials

2: $R_{\text{out}} \leftarrow R_{\text{out}} + \sum_{S,S' \subseteq [n]} c_{S,S'} (\prod_{i \in S} R_i^{\text{in}}) R_{S'}$

3: ToggleMonomials

4: $R_{\text{out}} \leftarrow R_{\text{out}} - \sum_{S,S' \subseteq [n]} c_{S,S'} (\prod_{i \in S} R_i^{\text{in}}) R_{S'}$

When Line2 executes, we have $R_{S'} = \tau_{S'} + \prod_{i \in S'} y_i$, so after that line,

$$R_{\text{out}} = \tau^{\text{out}} + \sum_{S,S' \subseteq [n]} c_{S,S'} \left(\prod_{i \in S} \tau_i^{\text{in}} \right) \left(\tau_{S'} + \prod_{i \in S'} y_i \right).$$

Then when Line4 executes, we have $R_{S'} = \tau_{S'}$, so the final value is

$$R_{\text{out}} = \tau^{\text{out}} + \sum_{S,S' \subseteq [n]} c_{S,S'} \left(\prod_{i \in S} \tau_i^{\text{in}} \right) \left(\prod_{i \in S'} y_i \right) = \tau^{\text{out}} + f(x_1, \ldots, x_n).$$

So Algorithm1 correctly computes f. The space of the program is $n + 2^n$ by construction, and as before we can ignore the instructions on lines2 and4 since they do not use $\mathbf{x}$, giving us a total runtime of $4n$ from the two calls to ToggleMonomials. □

3.2 Trading Space for Time

In this section, we will modify Algorithm1 to make m dramatically smaller, in exchange for making the branching program longer. Before jumping into the proof of Theorem11, we will address the main innovation of our work, namely trading off time for space. Namely we begin by building a register program that takes time $n2^{n+1}$ but uses only the $n+1$ registers $R_1^{\text{in}} \dots R_n^{\text{in}}, R_{\text{out}}$. This is similar to what Theorem11 guarantees when $k = n$. As in Sections3.1 and3.3, let $p_f \in \mathbb{F}_2[x_1, \dots, x_n]$ be f as a polynomial, let $q_f \in \mathbb{F}_2[\tau_1^{\text{in}}, \dots, \tau_n^{\text{in}}, y_1, \dots, y_n]$ be equal to p_f so long as $y_i = \tau_i^{\text{in}} + x_i$ for all i, and let $c_{S,S'}$ be the coefficient of $(\prod_{i \in S} \tau_i^{\text{in}})(\prod_{i \in S'} y_i)$ in q_f. We define a small generalization of ToggleInput, where we can choose to toggle only a subset of our inputs:

ToggleSomeInputs(S').

1: **procedure** ToggleSomeInputs(S')

2: **for** $i \in S'$ **do**

3: $R_i^{\text{in}} \leftarrow R_i^{\text{in}} + x_i$

4: **end for**

5: **end procedure**

Using ToggleSomeInputs(S'), we can replace the register $R_{S'}$ in Algorithm1 with a separate set of instructions that computes the corresponding term of q_f:

Algorithm2: A slow algorithm for computing q_f.

1: **for** $S' \subseteq [n]$ **do**

2: ToggleSomeInputs(S')

3: $R_{\text{out}} \leftarrow R_{\text{out}} + \sum_{S \subseteq [n]} c_{S,S'} \cdot \prod_{i \in S \cup S'} R_i^{\text{in}}$

4: ToggleSomeInputs(S')

5: **end for**

Whenever Line3 is executed, $R_i^{\text{in}} = y_i$ for $i \in S'$, and $R_i^{\text{in}} = \tau_i^{\text{in}}$ for $i \notin S'$. By construction of q_f, S and S' are disjoint whenever $c_{S,S'} \neq 0$, from which it follows that $R_i^{\text{in}} = \tau_i^{\text{in}}$ for $i \in S$. Thus the effect of Line3 is to add $\sum_{S \subseteq [n]} c_{S,S'}(\prod_{i \in S} \tau_i^{\text{in}})(\prod_{i \in S'} y_i)$ to R_{out}. Since this is run for every subset S', the overall effect of the program is to add

$$\sum_{S,S' \subseteq [n]} c_{S,S'} \left(\prod_{i \in S} \tau_i^{\text{in}} \right) \left(\prod_{i \in S'} y_i \right) = p_f(x_1 \dots x_n)$$

to R_{out}. We now give an overview of the full proof. Our goal is to balance Algorithms1 and2 by removing the registers R_S corresponding to large subsets S and using slow multiplication to build the polynomial q_f from the remaining small subsets. In particular, if we divide the input bits into k groups each of size $\lceil n/k \rceil$, and only store all subsets within each group, then any monomial

$c_{S,S'} \prod_{i \in S} \tau_i^{\text{in}} \prod_{i \in S'} y_i$ can be computed by multiplying together one subset from each group, namely the restriction of S to the group. Instead of 2^n registers for all subsets, we use only $k \cdot 2^{\lceil n/k \rceil}$ registers corresponding to subsets in the k groups, and we can compute all the corresponding monomials into these registers in time $2n$ using the first half of Algorithm1. Then since we are only multiplying k monomials together, we can compute q_f using Algorithm2 in time $2^k \cdot 2 \cdot 2n$, since each call to ToggleSomeInputs is replaced with our $2n$ time execution of Algorithm1. As a slight last speedup, we use a Gray code to order our subsets in Algorithm2, replacing two executions of ToggleSomeInputs with a subroutine toggling a single group on or off and only spending $2\lceil n/k \rceil$ time to do so. This allows us to compute q_f in $2^k \cdot 2\lceil n/k \rceil$ time rather than $2^k \cdot 4n$ time.

Proof of Theorem11. For $j \in [k]$ let $b_j = \lceil nj/k \rceil$, and divide the range $[n]$ into k groups: $G_1 = \{1, \ldots, b_1\}$, $G_2 = \{b_1 + 1, \ldots, b_2\}$, ..., $G_k = \{b_{k-1} + 1, \ldots, b_n = n\}$. For each group G_j, we have $2^{|G_j|}$ registers $R_{j,S}$ indexed by subsets $S \subseteq G_j$. As in all previous algorithms we also use n registers $R_1^{\text{in}}, \ldots, R_n^{\text{in}}$, corresponding to the n input bits, for a total of $n + \sum_{j=1}^{k} 2^{|G_j|}$ registers plus the output register R_{out}. We'll begin by rewriting the polynomial q_f in a new form. Recall from Section3.1 that $q_f(\vec{\tau}^{\text{in}}, \vec{y}) = f(x)$ so long as $\vec{y} = \vec{x} + \vec{\tau}^{\text{in}}$, and

$$q_f(\vec{\tau}^{\text{in}}, \vec{y}) = \sum_{S, S' \subseteq [n]} c_{S,S'} \left(\prod_{i \in S} \tau_i^{\text{in}} \right) \left(\prod_{i \in S'} y_i \right).$$

Now, for every $S' \subseteq [n]$, define $S'_j := S' \cap G_j$. For each $j \in [k]$ and $S \subseteq G_j$, let $z_{j,S} = \tau_{j,S} + \prod_{i \in S} y_i$, where $\tau_{j,S}$ is the initial value of register $R_{j,S}$. Now for every monomial in q_f, we split the term $\prod_{i \in S'} y_i$ in the monomial into k different products $\prod_{i \in S'_j} y_i$, each of which we can replace with $z_{j,S'_j} - \tau_{j,S'_j}$. This gives us a new polynomial

$$r_f(\vec{\tau}^{\text{in}}, \vec{\tau}, \vec{z}) = \sum_{S, S' \subseteq [n]} c_{S,S'} \left(\prod_{i \in S} \tau_i^{\text{in}} \right) \left(\prod_{j=1}^{k} (z_{j,S'} - \tau_{j,S'}) \right).$$

As we did with q_f, for $S, S' \subseteq [n]$ and $T \subseteq [k]$, let $d_{S,S',T}$ be the coefficient of $(\prod_{i \in S} \tau_i^{\text{in}}) \cdot (\prod_{j \in T} z_{j,S'_j}) \cdot (\prod_{j \in [k] \setminus T} \tau_{j,S'_j})$ in r_f, so that

$$r_f(\vec{\tau}^{\text{in}}, \vec{\tau}, \vec{z}) = \sum_{S \subseteq [n]} \sum_{S' \subseteq [n]} \sum_{T \subseteq [k]} d_{S,S',T} \cdot \left(\prod_{i \in S} \tau_i^{\text{in}} \right) \left(\prod_{j \in T} z_{j,S'_j} \right) \left(\prod_{j \in [k] \setminus T} \tau_{j,S'_j} \right)$$

which is equivalent to $f(x_1 \ldots f_n)$ as long as $z_{j,S'_j} = \tau_{j,S'_j} + \prod_{i \in S'_j} (x_i + \tau_i^{\text{in}} + 1)$. Following ToggleSomeInputs(S'), we define new versions of ToggleInput and ToggleMonomials from Section3.1 which focus on some groups and not others. In fact we will only focus on a single group G_j rather than a subset of the groups, as we will order our subsets S' in such a way that we will only ever need to toggle one group at a time:

ToggleInputForGroup(j).

1: **procedure** ToggleInputForGroup(j)

2: **for** $i \in G_j$ **do**

3: $R_i^{\text{in}} \leftarrow R_i^{\text{in}} + x_i$

4: **end for**

5: **end procedure**

ToggleMonomialsForGroup(j).

1: **procedure** ToggleMonomialsForGroup(j)

2: ToggleInputForGroup(j)

3: **for** $S \subseteq G_j$ **do**

4: $R_{j,S} \leftarrow R_{j,S} + \prod_{i \in S} R_i^{\text{in}}$

5: **end for**

6: ToggleInputForGroup(j)

7: **end procedure**

We are now ready to assemble Algorithm4, which completes the proof of Theorem11. To incorporate our improvement using Gray codes[9], let $T_0 = \emptyset, \ldots, T_{2^k-1}$ be an ordering of all subsets of $[k]$ such that each consecutive pair of sets $T_\ell, T_{\ell+1 \bmod 2^k}$ differs by exactly one element $e_\ell \in [k]$; thus we will only need to toggle the group G_{e_ℓ} as claimed:

Algorithm3: Transparently compute f in time $2^k \cdot 2\lceil n/k \rceil$ with $n + k \cdot 2^{\lceil n/k \rceil}$ registers.

1: **for** $\ell = 0, \ldots, 2^k - 1$ **do**

2:
$$R_{\text{out}} \leftarrow R_{\text{out}} + \sum_{S \subseteq [n]} \sum_{S' \subseteq [n]} d_{S,S',T_\ell} \left(\prod_{i \in S} R_i^{\text{in}}\right)\left(\prod_{j=1}^{k} R_{j,S'_j}\right)$$

3: ToggleMonomialsForGroup(e_ℓ)

4: **end for**

Each time Line2 is reached, we have $R_{j,S} = \tau_{j,S} + \prod_{i \in S} y_j$ for $j \in T_\ell$, and $R_{j,S} = \tau_{j,S}$ for $j \in [k] \setminus T_\ell$. We also have $R_i^{\text{in}} = \tau_i^{\text{in}}$ for each $i \in [n]$. So the effect of the line is to add

$$\sum_{S \subseteq [n]} \sum_{S' \subseteq [n]} d_{S,S',T_\ell} \left(\prod_{i \in S} \tau_i^{\text{in}}\right)\left(\prod_{j \in T_\ell} z_{j,S'_j}\right)\left(\prod_{j \in [k] \setminus T_\ell} \tau_{j,S'_j}\right)$$

to R_{out}. Summing this expression over all possible subsets $T_\ell \subseteq [k]$ gives $R_{\text{out}} = \tau^{\text{out}} + r_f(\cdots) = \tau^{\text{out}} + f(x_1, \ldots, x_n)$, and so our algorithm transparently computes f. □

Note12. It is not difficult to save k registers by removing $R_{1,\emptyset}, \ldots, R_{k,\emptyset}$, as we simply add each value $d_{S,\emptyset,T}(\prod_{i\in S} R_i^{\text{in}})$ to our polynomial without concerning ourselves with any x_i (and by extension any y_i or z_i) variables.

3.3 Bounded-Degree Polynomials

Robere and Zuiddam [12,Theorem5.13] showed that if f is a polynomial of degree $d < n$, it is possible to improve on Potechin's theorem by decreasing $m = 2^{2^n-1}$ down to $m = 2^{\binom{n}{\leq d}-1}$. Here we show how to adapt Algorithm1 to get a similar result, and then at the end of the section we build a tradeoff algorithm to improve it.

Theorem13. For any function $f : \{0,1\}^n \to \{0,1\}$ which is a degree-d polynomial, there is an m-catalytic branching program with length $4n$ and width $2m$ that computes f, where $m \leq 2^{n+\binom{n}{\leq d}}$. Again, while this is slightly worse than Robere and Zuiddam's original result, we include it to show the flexibility of our approach and as a stepping stone to our tradeoff result.

Proof. The proof can be summarized as follows. We start with Algorithm1, and make the following change: for every $S' \in [n]$ such that $c_{S,S'} = 0$ for all S, remove the register $R_{S'}$. We then argue that only $n + \binom{n}{\leq d}$ registers remain. Let $p_f = f \in \mathbb{F}_2[x_1, \ldots, x_n]$ – since f is already a polynomial, there's no need to define p_f using interpolation this time. As before, let

$$q_f(\vec{\tau}^{\text{in}}, \vec{y}) = p_f(\vec{y} - \vec{\tau}^{\text{in}}) = \sum_{S,S'\subseteq[n]} c_{S,S'} \left(\prod_{i\in S} \tau_i^{\text{in}}\right)\left(\prod_{i\in S'} y_i\right).$$

Let $M_f \subseteq 2^{[n]}$ be the set of all S' such that there exists an S where $c_{S,S'} \neq 0$. We define the following subroutine:

ToggleUsefulMonomials.

1: **procedure** ToggleUsefulMonomials

2: ToggleInput

3: **for** $S \in M_f$ **do**

4: $R_S \leftarrow R_S + \prod_{i\in S} R_i^{\text{in}}$

5: **end for**

6: ToggleInput

7: **end procedure**

The only difference from ToggleMonomials is that we ignore subsets S which are not in M_f (not "useful"). Our final algorithm is

Algorithm4: Transparently compute a degree-d polynomial f in time $4n$ with $n + \binom{n}{\leq d}$ registers.

1: ToggleUsefulMonomials

2: $R_{\text{out}} \leftarrow R_{\text{out}} + \sum_{S \subseteq [n]} \sum_{S' \in M_f} c_{S,S'} (\prod_{i \in S} R_i^{\text{in}}) R_{S'}$

3: ToggleUsefulMonomials

4: $R_{\text{out}} \leftarrow R_{\text{out}} - \sum_{S \subseteq [n]} \sum_{S' \in M_f} c_{S,S'} (\prod_{i \in S} R_i^{\text{in}}) R_{S'}$

To conclude the proof of Theorem13, we need to show $|M_f| \leq \binom{n}{\leq d}$. Indeed, since p_f is a degree-d polynomial, q_f also has degree d, which means $c_{S,S'} = 0$ whenever $|S| + |S'| > d$. So, M_f only contains sets S' with size at most d, of which there are $\binom{n}{\leq d}$. □

Note14. The original algorithms of [11,12] rely on the symmetries of f as an $\mathbb{F}_2$ polynomial, in essence having each start state represent a possible function g which can be obtained from f by negating input variables or taking $\oplus$ with f itself. [11] takes this set of functions to be the space of all n-variable functions, while [12] analyzes these rules and obtains a more exact characterization. While this characterization is phrased in terms of orbit closures, it can also be described in terms of polynomials as the span of the set of all monomials appearing in f as an $\mathbb{F}_2$ polynomial along with all submonomials of this set; this exactly coincides with our notion as $\prod_{i \in S} y_i$ generates all submonomials $\prod_{i \in S' \subseteq S} x_i$ for $y_i := x_i + \tau_i$, which leads to the quantitative results being essentially the same despite taking two completely different approaches. Now we state our tradeoff algorithm, which goes much in the same way as Theorem11 but without breaking the variables into groups.

Proof of Theorem15. For any $\Delta \in \mathbb{N}$, let $M_f^{\Delta} \subseteq \binom{n}{\leq \Delta}$ be the set of all S'' of size at most Δ such that there exists an $S \subseteq [n]$ and $S' \supseteq S''$ where $c_{S,S'} \neq 0$. We will have k registers $R_{j,S''}$ for every $S'' \in M_f^{\lceil d/k \rceil}$, as well as the usual registers $R_1^{\text{in}} \dots R_n^{\text{in}}, R_{\text{out}}$. Note that this gives us our target space, as $|M_f^{\lceil d/k \rceil}| \leq \binom{n}{\leq \lceil d/k \rceil}$. Following our proof of Theorem11, let $z_{j,S} = \tau_{j,S} + \prod_{i \in S} y_i$, where $\tau_{j,S}$ is the initial value of register $R_{j,S}$, and for every monomial in q_f we split the term $\prod_{i \in S'} y_i$ in the monomial arbitrarily into k different products $\prod_{i \in S'_j} y_i$ – each of which we can replace with $z_{j,S'_j} - \tau_{j,S'_j}$ – where $S'_j \in M_f^{\lceil d/k \rceil}$ and $\cup_{j \in [k]} S'_j = S'$. This is possible because each non-zero term in q_f has degree at most d, meaning that $|S'| \leq d$ and furthermore every subset of S' of size at most $\lceil d/k \rceil$ appears in $M_f^{\lceil d/k \rceil}$ by construction.[4] Fixing some particular partition $(S'_j)_{j \in [k]}$ for each

[4]Our use of j here is slightly different than in our previous proof; namely, j is not linked to a specific block of variables, and rather we arbitrarily partitioned S' into k sets and assigned them each a distinct j. This will result in us having to spend time n to load the monomials in, rather than time $\lceil n/k \rceil$ as in the previous proof, but this is necessary as we have no guarantee that there is a partition of the variables such that every monomial of degree at most d is split into k monomials of degree at most $\lceil d/k \rceil$. Note that this is where our algorithm performs worse than Algorithm4 when $d = \Omega(n)$.

S', this gives us a new polynomial

$$r_f(\vec{\tau}^{\text{in}}, \vec{\tau}, \vec{z}) = \sum_{S\subseteq[n]} \sum_{S'\subseteq[n]} \sum_{T\subseteq[k]} d_{S,S',T} \cdot \left(\prod_{i\in S} \tau_i^{\text{in}}\right) \left(\prod_{j\in T} z_{j,S'_j}\right) \left(\prod_{j\in[k]\setminus T} \tau_{j,S'_j}\right)$$

which is equivalent to $f(x_1 \dots f_n)$ as long as $z_{j,S'_j} = \tau_{j,S'_j} + \prod_{i\in S'_j}(x_i + \tau_i^{\text{in}} + 1)$. We define ToggleMonomialsForGroup as before, using ToggleInput instead of ToggleInputForGroup since the variables are no longer split into groups, and using a Gray code we get our final algorithm:

Algorithm5: Transparently compute f in time $2^k \cdot 2n$ with $n + k \cdot \binom{n}{\leq \lceil d/k \rceil}$ registers.

1: **for** $\ell = 0, \dots, 2^k - 1$ **do**

2: $$R_{\text{out}} \leftarrow R_{\text{out}} + \sum_{S\subseteq[n]} \sum_{S'\subseteq[n]} d_{S,S',T_\ell} (\prod_{i\in S} R_i^{\text{in}})(\prod_{j=1}^{k} R_{j,S'_j})$$

3: ToggleMonomialsForGroup(e_ℓ)

4: **end for**

The analysis is identical to that of Algorithm4, except the runtime is $2^k \cdot 2n$ rather than $2^k \cdot 2\lceil n/k \rceil$ because we do not split the variables into groups. □

References

[1] Eric Allender, Anna Gál, and Ian Mertz. Dual VP classes. Comput. Complex., 26(3):583–625, 2017.

[2] David A. Mix Barrington. Bounded-width polynomial-size branching programs recognize exactly those languages in nc1. J. Comput. Syst. Sci., 38(1):150–164, 1989.

[3] Michael Ben-Or and Richard Cleve. Computing algebraic formulas using a constant number of registers. SIAM J. Comput., 21(1):54–58, 1992.

[4] Harry Buhrman, Richard Cleve, Michal Koucký, Bruno Loff, and Florian Speelman. Computing with a full memory: catalytic space. In Symposium on Theory of Computing, STOC 2014, New York, NY, USA, May 31 - June 03, 2014, pages 857–866. ACM, 2014.

[5] James Cook and Ian Mertz. Catalytic approaches to the tree evaluation problem. In Proccedings of the 52nd Annual ACM SIGACT Symposium on Theory of Computing, STOC 2020, Chicago, IL, USA, June 22-26, 2020, pages 752–760. ACM, 2020.

[6] James Cook and Ian Mertz. Encodings and the tree evaluation problem. Electron. Colloquium Comput. Complex., page 54, 2021. URL: https://eccc.weizmann.ac.il/report/2021/054.

[7] Stephen A. Cook, Pierre McKenzie, Dustin Wehr, Mark Braverman, and Rahul Santhanam. Pebbles and branching programs for tree evaluation. ACM Trans. Comput. Theory, 3(2):4:1– 4:43, 2012.

[8] Vincent Girard, Michal Koucky, and Pierre McKenzie. Nonuniform catalytic space and the direct sum for space. Electronic Colloquium on Computational Complexity (ECCC), 138, 2015.

[9] Frank Gray. Pulse code communication. https://patents.google.com/patent/US2632058A/ en, 1953. US Patent 2632058A.

[10] E.I. Nečiporuk. A boolean function. Dokl. Akad. Nauk SSSR, 169(4), 1966.

[11] Aaron Potechin. A note on amortized branching program complexity, 2017.

[12] Robert Robere and Jeroen Zuiddam. Amortized circuit complexity, formal complexity measures, and catalytic algorithms. In FOCS, pages 759–769. IEEE, 2021.